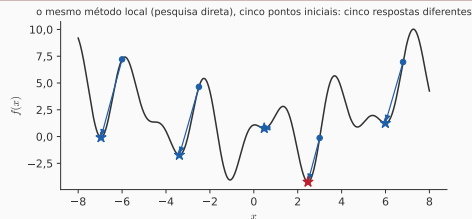


Otimização

5.1 GLODS — *Global and Local Optimization using Direct Search*

José Firmino Aguilar Madeira

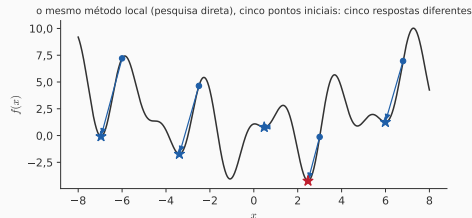
Capítulo 5. Otimização global — local ou global?



$f(x) = 0,1x^2 + 3\sin 2x + 2\cos(3,3x + 1)$ em $X = [-8, 8]$: a função de referência de todo o capítulo.

O mesmo método local, cinco pontos iniciais: **os**
cinco convergiram.

Capítulo 5. Otimização global — local ou global?

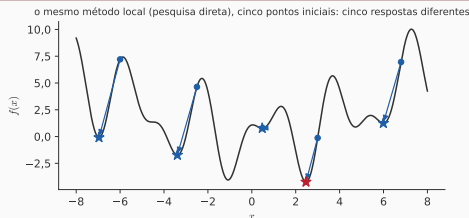


$f(x) = 0,1x^2 + 3\sin 2x + 2\cos(3,3x + 1)$ em $X = [-8, 8]$: a função de referência de todo o capítulo.

O mesmo método local, cinco pontos iniciais: **os cinco convergiram.**

Pergunta: qual deles resolveu o problema?

Capítulo 5. Otimização global — local ou global?



$f(x) = 0,1x^2 + 3\sin 2x + 2\cos(3,3x + 1)$ em $X = [-8, 8]$: a função de referência de todo o capítulo.

O mesmo método local, cinco pontos iniciais: **os cinco convergiram.**

Pergunta: qual deles resolveu o problema?

mínimo **local**: $f(\mathbf{x}^*) \leq f(\mathbf{x})$ para $\mathbf{x} \in X$,
 $\|\mathbf{x} - \mathbf{x}^*\| < \delta$ (algum $\delta > 0$)

mínimo **global**:

$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \text{ para todo o } \mathbf{x} \in X$$

Todo o mínimo global é local; o contrário não.

Convergir não significa encontrar a melhor solução do problema.

Porque interessa procurar globalmente?

Até aqui (caps. 2–4) aprendemos a **descer e convergir**. Num problema de engenharia isso não chega:

1. **Dependência do ponto inicial:** uma única pesquisa local pode devolver soluções diferentes consoante $\mathbf{x}_0$.

Porque interessa procurar globalmente?

Até aqui (caps. 2–4) aprendemos a **descer e convergir**. Num problema de engenharia isso não chega:

1. **Dependência do ponto inicial:** uma única pesquisa local pode devolver soluções diferentes consoante $\mathbf{x}_0$.
2. **Não sabemos quantas bacias existem:** numa caixa negra nem sequer vemos a paisagem.

Porque interessa procurar globalmente?

Até aqui (caps. 2–4) aprendemos a **descer e convergir**. Num problema de engenharia isso não chega:

1. **Dependência do ponto inicial:** uma única pesquisa local pode devolver soluções diferentes consoante $\mathbf{x}_0$.
2. **Não sabemos quantas bacias existem:** numa caixa negra nem sequer vemos a paisagem.
3. **Uma solução local pode ser muito pior do que outra:** ser estacionário não é, necessariamente, o que o engenheiro quer.

Porque interessa procurar globalmente?

Até aqui (caps. 2–4) aprendemos a **descer e convergir**. Num problema de engenharia isso não chega:

1. **Dependência do ponto inicial:** uma única pesquisa local pode devolver soluções diferentes consoante $\mathbf{x}_0$.
2. **Não sabemos quantas bacias existem:** numa caixa negra nem sequer vemos a paisagem.
3. **Uma solução local pode ser muito pior do que outra:** ser estacionário não é, necessariamente, o que o engenheiro quer.
4. **Vários bons mínimos também são informação:** fabrico, custo, robustez ou manutenção podem levar a escolher uma solução que não é a de menor f — voltamos a isto no fim do 5.1.

Mas há um preço

Se não sabemos onde está a melhor bacia, temos de **explorar**. E explorar **custa avaliações de f** .

Exploração

visitar regiões novas de X , à procura de outras bacias (*exploration*)

Refinamento

descer dentro de uma bacia já encontrada, até ao seu mínimo (*exploitation*)

Pergunta: como explorar diferentes bacias **sem gastar todo o orçamento?**

Mas há um preço

Se não sabemos onde está a melhor bacia, temos de **explorar**. E explorar **custa avaliações de f** .

Exploração

visitar regiões novas de X , à procura de outras bacias (*exploration*)

Refinamento

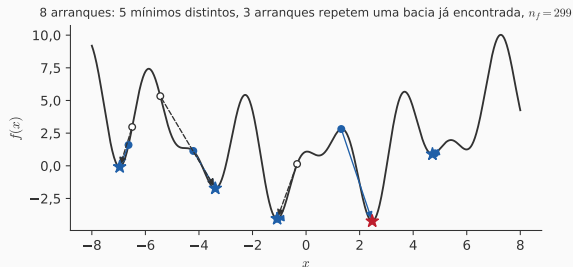
descer dentro de uma bacia já encontrada, até ao seu mínimo (*exploitation*)

Pergunta: como explorar diferentes bacias **sem gastar todo o orçamento?**

O objetivo deste capítulo

Procurar globalmente: explorar diferentes bacias e tentar identificar soluções melhores do que a de uma só pesquisa local. **Garantir** o ótimo global é outra questão.

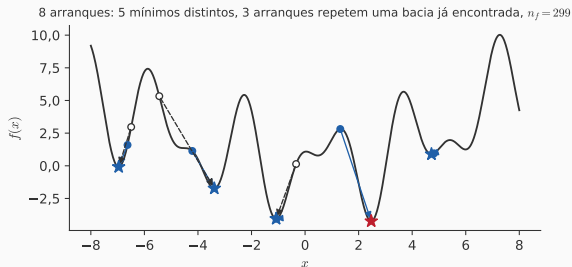
O remédio mais simples: multiarranque



Multiarranque: correr o mesmo método local a partir de vários pontos e ficar com o melhor.

Aqui, 8 arranques aleatórios encontraram 5 mínimos distintos, incluindo o global.

O remédio mais simples: multiarranque



Multiarranque: correr o mesmo método local a partir de vários pontos e ficar com o melhor.
Aqui, 8 arranques aleatórios encontraram 5 mínimos distintos, incluindo o global.

Pergunta: qual é o problema?

Três dos oito arranques (a tracejado) caíram numa bacia já encontrada e **repetiram praticamente o mesmo trabalho**.

Precisamos de explorar o espaço sem desperdiçar demasiado orçamento.

Três maneiras de escapar — o mapa do capítulo 5

Como escapamos de uma bacia local — e quanto nos custa fazê-lo?

5.1 GLODS

Vários pontos + pesquisas locais (pesquisa direta, 3.2.5) + evitar redundância.
Determinístico (com *search* determinístico); devolve uma *lista* de mínimos.

5.2 *Simulated annealing*

Um ponto que aceita, por vezes, *piorar*.
Estocástico.

5.3 Algoritmos genéticos

Uma população: seleção + recombinação + mutação.
Estocástico.

No fim de cada método, as mesmas cinco perguntas:

Exploração global
como escapa às
bacias?

Informação
o que precisa de
f?

Custo
quantas
avaliações?

Garantia
o que se pode
afirmar?

Aleatoriedade
uma corrida é
reproduzível?

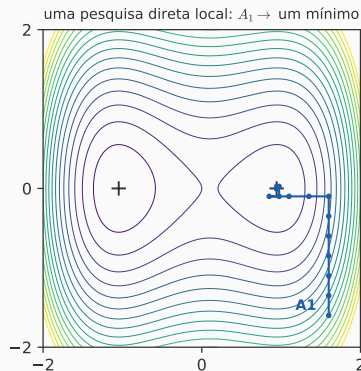
É o fio condutor da UC — informação, custo, garantia — aplicado à pergunta global.

GLODS: multiarranque condicionado pela proximidade

GLODS

Uma estratégia multiarranque **condicionada pela proximidade**: se duas pesquisas ficam suficientemente próximas, **só uma permanece ativa**.

- Uma **pesquisa direta** (3.2.5) de A_1 tende a convergir para o mínimo da sua bacia.



Aluffi-Pentini em $[-2, 2]^2$ (a sequência da apresentação de 2015, refeita).

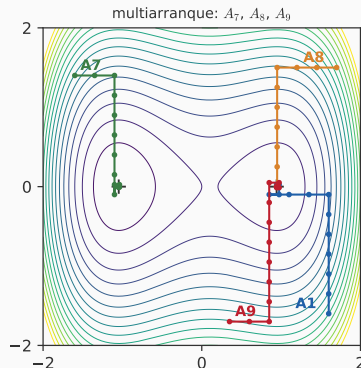
Só valores de f ; f cara; X compacto (barreira extrema). Custódio e Madeira, *J. Global Optim.* 62(1):1–28, 2015.

GLODS: multiarranque condicionado pela proximidade

GLODS

Uma estratégia multiarranque **condicionada pela proximidade**: se duas pesquisas ficam suficientemente próximas, **só uma permanece ativa**.

- Uma **pesquisa direta** (3.2.5) de A_1 tende a convergir para o mínimo da sua bacia.
- **Multiarranque**: pesquisas de A_7, A_8, A_9 , cada uma até ao fim.



Aluffi-Pentini em $[-2, 2]^2$ (a sequência da apresentação de 2015, refeita).

Só valores de f ; f cara; X compacto (barreira extrema). Custódio e Madeira, *J. Global Optim.* 62(1):1–28, 2015.

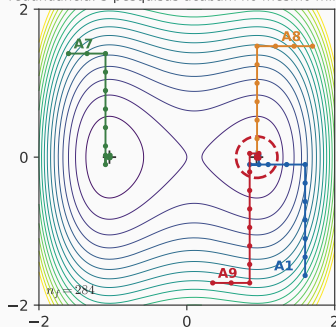
GLODS: multiarranque condicionado pela proximidade

GLODS

Uma estratégia multiarranque **condicionada pela proximidade**: se duas pesquisas ficam suficientemente próximas, **só uma permanece ativa**.

- Uma **pesquisa direta** (3.2.5) de A_1 tende a convergir para o mínimo da sua bacia.
- **Multiarranque**: pesquisas de A_7 , A_8 , A_9 , cada uma até ao fim.
- **Redundância**: três acabam no mesmo mínimo; $n_f = 284$ para 2 mínimos.

redundância: 3 pesquisas acabam no mesmo mínimo



Aluffi-Pentini em $[-2, 2]^2$ (a sequência da apresentação de 2015, refeita).

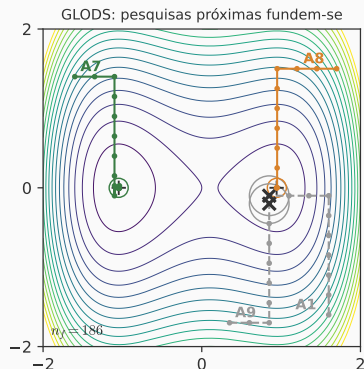
Só valores de f ; f cara; X compacto (barreira extrema). Custódio e Madeira, *J. Global Optim.* 62(1):1–28, 2015.

GLODS: multiarranque condicionado pela proximidade

GLODS

Uma estratégia multiarranque **condicionada pela proximidade**: se duas pesquisas ficam suficientemente próximas, **só uma permanece ativa**.

- Uma **pesquisa direta** (3.2.5) de A_1 tende a convergir para o mínimo da sua bacia.
- **Multiarranque**: pesquisas de A_7 , A_8 , A_9 , cada uma até ao fim.
- **Redundância**: três acabam no mesmo mínimo; $n_f = 284$ para 2 mínimos.
- **GLODS**: as mesmas pesquisas em simultâneo; a que entra na vizinhança de outra melhor deixa de estar ativa ($A_9 \rightarrow A_1$, $A_1 \rightarrow A_8$). Os mesmos 2 mínimos, $n_f = 186$.



Aluffi-Pentini em $[-2, 2]^2$ (a sequência da apresentação de 2015, refeita).

Só valores de f ; f cara; X compacto (barreira extrema). Custódio e Madeira, *J. Global Optim.* 62(1):1–28, 2015.

A ideia que distingue o GLODS: $(\mathbf{x}, \alpha, r, i)$

Cada ponto da lista guarda quatro coisas:

$\mathbf{x}$ posição

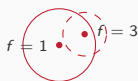
α passo da pesquisa direta

r raio de comparação

i ativo / inativo



longe: duas pesquisas ativas



próximas: um cai no raio do outro



só a melhor fica ativa

Regra (decréscimo simples)

Um ponto novo $\mathbf{x}_{\text{new}}$ é **comparável** com um ponto armazenado $\mathbf{y}$ quando $\|\mathbf{x}_{\text{new}} - \mathbf{y}\| \leq r_y$. Não comparável com ninguém: região nova, entra **ativo**. Melhor do que todos os comparáveis: entra **ativo** e inativa os piores. Algum comparável tão bom ou melhor: entra **inativo** (se é melhor do que algum ativo) ou nem entra. Um ativo pode passar a inativo; o contrário nunca. Só os ativos são centros de *poll*.

O raio vem do **passo da pesquisa direta**, não de uma construção probabilística: $r = \text{passo com que o ponto entrou}$ ($\alpha \|\mathbf{d}\|$; α com direções unitárias); não diminui quando α diminui ($r \geq \alpha$).

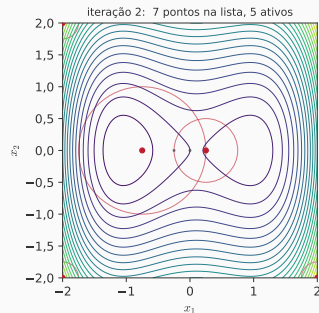
Quando duas pesquisas se aproximam: fusão

O GLODS completo, com *search* por 2^n -centros, no mesmo problema: os pontos novos passam pela regra de comparação e as pesquisas que se aproximam de outras melhores deixam de estar ativas.

Três tipos de iteração

Bem sucedida: entrou um ativo novo — o passo mantém-se ou aumenta (nos exemplos, $\times 2$). **Sem sucesso:** não entrou ninguém — o passo do centro reduz-se (nos exemplos, $\times \frac{1}{2}$).

Fusão: só entraram inativos — não é falhanço: a proximidade detetou trabalho redundante.



Aluffi-Pentini, $\alpha_0 = r_0 = 0,25$. Vermelho: ativos, com o raio; cinzento: inativos.

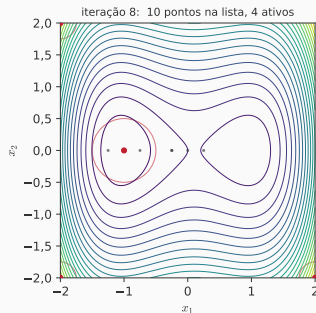
Quando duas pesquisas se aproximam: fusão

O GLODS completo, com *search* por 2^n -centros, no mesmo problema: os pontos novos passam pela regra de comparação e as pesquisas que se aproximam de outras melhores deixam de estar ativas.

Três tipos de iteração

Bem sucedida: entrou um ativo novo — o passo mantém-se ou aumenta (nos exemplos, $\times 2$). **Sem sucesso:** não entrou ninguém — o passo do centro reduz-se (nos exemplos, $\times \frac{1}{2}$).

Fusão: só entraram inativos — não é falhanço: a proximidade detetou trabalho redundante.



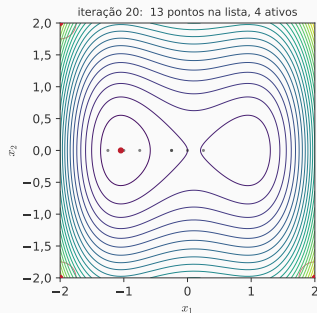
Aluffi-Pentini, $\alpha_0 = r_0 = 0,25$. Vermelho: ativos, com o raio; cinzento: inativos.

Quando duas pesquisas se aproximam: fusão

O GLODS completo, com *search* por 2^n -centros, no mesmo problema: os pontos novos passam pela regra de comparação e as pesquisas que se aproximam de outras melhores deixam de estar ativas.

Três tipos de iteração

Bem sucedida: entrou um ativo novo — o passo mantém-se ou aumenta (nos exemplos, $\times 2$). **Sem sucesso:** não entrou ninguém — o passo do centro reduz-se (nos exemplos, $\times \frac{1}{2}$). **Fusão:** só entraram inativos — não é falhanço: a proximidade detetou trabalho redundante.



Aluffi-Pentini, $\alpha_0 = r_0 = 0,25$. Vermelho: ativos, com o raio; cinzento: inativos.

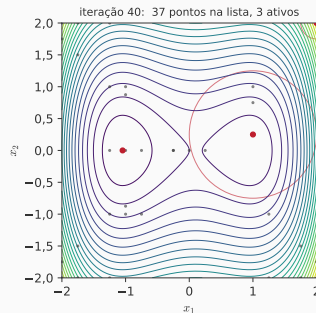
Quando duas pesquisas se aproximam: fusão

O GLODS completo, com *search* por 2^n -centros, no mesmo problema: os pontos novos passam pela regra de comparação e as pesquisas que se aproximam de outras melhores deixam de estar ativas.

Três tipos de iteração

Bem sucedida: entrou um ativo novo — o passo mantém-se ou aumenta (nos exemplos, $\times 2$). **Sem sucesso:** não entrou ninguém — o passo do centro reduz-se (nos exemplos, $\times \frac{1}{2}$).

Fusão: só entraram inativos — não é falhanço: a proximidade detetou trabalho redundante.



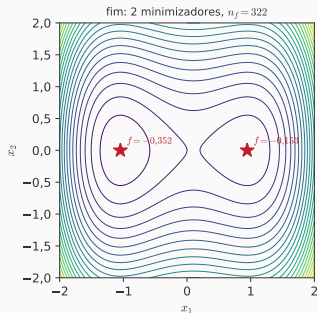
Aluffi-Pentini, $\alpha_0 = r_0 = 0,25$. Vermelho: ativos, com o raio; cinzento: inativos.

Quando duas pesquisas se aproximam: fusão

O GLODS completo, com *search* por 2^n -centros, no mesmo problema: os pontos novos passam pela regra de comparação e as pesquisas que se aproximam de outras melhores deixam de estar ativas.

Três tipos de iteração

Bem sucedida: entrou um ativo novo — o passo mantém-se ou aumenta (nos exemplos, $\times 2$). **Sem sucesso:** não entrou ninguém — o passo do centro reduz-se (nos exemplos, $\times \frac{1}{2}$). **Fusão:** só entraram inativos — não é falhanço: a proximidade detetou trabalho redundante.

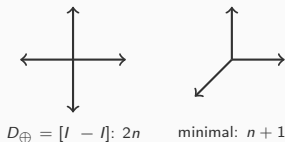


Aluffi-Pentini, $\alpha_0 = r_0 = 0,25$. Vermelho: ativos, com o raio; cinzento: inativos. Fim: 2 minimizantes, $n_f = 322$; 32 it. bem-sucedidas, 29 sem sucesso, 10 de fusão. Multiarranque de 8 pontos aleatórios: numa corrida, os mesmos 2 ($n_f = 848$); em 30 corridas, $n_f = 850$ [823; 882] e os 2 só em 12.

Poll: a componente local (pesquisa direta, 3.2.5)

A partir de $\mathbf{x}_k$ com passo α_k , avaliar f nos pontos de **sondagem** (poll) $\mathbf{x}_k + \alpha_k \mathbf{d}$, $\mathbf{d} \in D_k$, em que D_k é um **conjunto gerador positivo**: qualquer vetor de $\mathbb{R}^n$ é combinação linear *não negativa* de elementos de D_k . Se $\nabla f(\mathbf{x}_k) \neq \mathbf{0}$, alguma direção de D_k é de descida — sem calcular o gradiente.

- Base positiva maximal $D_{\oplus} = [I \ -I]$: $2n$ direções ($\pm \mathbf{e}_i$); minimal: $n + 1$ direções.
- **Sucesso** (algun ponto de sondagem melhora): o passo mantém-se ou cresce, $\alpha_{k+1} \in [\alpha_k, \gamma \alpha_k]$; **insucesso**: decresce, $\alpha_{k+1} \in [\beta_1 \alpha_k, \beta_2 \alpha_k]$, $0 < \beta_1 \leq \beta_2 < 1$. Nos exemplos: $\times 2$ e $\times \frac{1}{2}$.
- Limites $\mathbf{x}^{(L)} \leq \mathbf{x} \leq \mathbf{x}^{(U)}$ por *barreira extrema*: $f = +\infty$ fora de X (não se avalia; não confundir com a barreira logarítmica do 4.3.2).
- Paragem local: $\alpha_k \leq \alpha_{\min}$. O passo de sondagem é o α_k dos caps. 2–3 e o P_j do Hooke–Jeeves; volta em 6.5 (DMS).



Custo por iteração

$n_f \leq |D_k|$ ($2n$ com $D_{\oplus}$); só valores de f . É o “termostato” do Hooke–Jeeves e do Box; nas versões formalizadas (GPS, 3.2.5; aqui no GLODS) — não no Box simplificado do 3.2.4 —, dá resultados de estacionariedade sob hipóteses adequadas, que o NM não tem.

Search: a componente global

Como nos métodos de pesquisa direta modernos (patternsearch), a iteração organiza-se em dois passos:

Search (opcional, global)

Avaliar f num conjunto finito de pontos *qualquer* do domínio, à procura de regiões promissoras ainda não exploradas: amostragem aleatória, hipercubo latino, sequências de Sobol/Halton, ou 2^n -centros (página seguinte). *Se gerar um novo ponto ativo, a iteração é bem-sucedida e termina sem poll.* Não é preciso fazê-lo em todas as iterações: no artigo, o *search* executa-se quando o número de pontos ativos ainda não identificados como minimizantes locais é igual a 1.

Poll (local, estruturado)

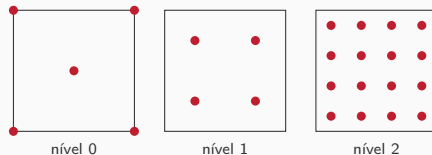
Explora uma região ativa: a sondagem $\mathbf{x}_k + \alpha_k \mathbf{d}$, $\mathbf{d} \in D_k$, em torno do ponto ativo escolhido como centro (o de menor f ainda por explorar). É este passo, com a estrutura dos conjuntos geradores positivos e do passo α_k , que *sustenta a teoria de convergência*; o *search* pode ser qualquer coisa.

Cada ponto novo, venha do *search* ou do *poll*, passa pela mesma regra de comparação («A ideia que distingue o GLODS»): é ela que decide se entra ativo, inativo, ou nem entra.

Uma possibilidade para o *search*: 2^n -centros

Uma estratégia **determinística** que cumpre três requisitos: pontos assintoticamente densos em X , cobertura uniforme e reprodutibilidade.

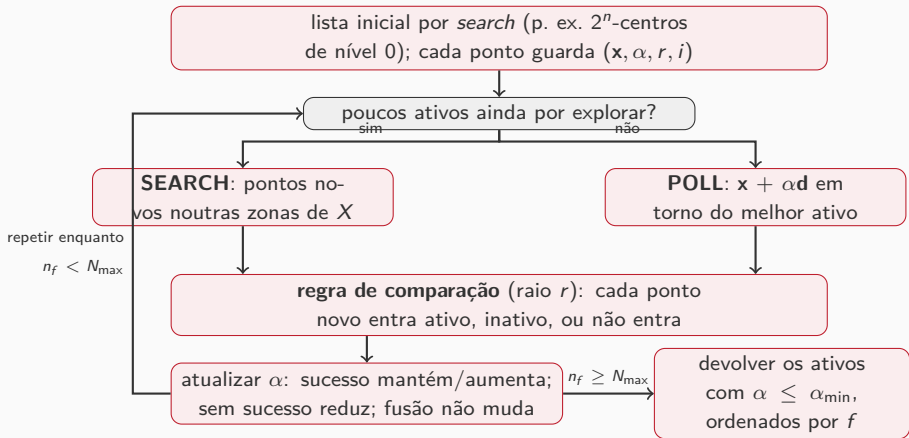
- **Nível 0**: o centro e os *vértices* da caixa ($2^n + 1$ pontos).
- **Nível $\ell \geq 1$** : a caixa é subdividida em $(2^\ell)^n$ caixas; os pontos são os seus centros.
- Para não explodir o custo, cada *search* usa só um subconjunto de 2^n centros ainda não usados; os restantes ficam para os seguintes.
- Cada centro novo passa pela regra de comparação (os que caem no raio de um ponto melhor entram inativos ou nem entram). Na versão didática entram com α_0, r_0 ; no algoritmo completo, pontos comparáveis podem herdar os parâmetros da pesquisa existente.



$n = 2$: nível 0, 5 pontos (centro + vértices); nível 1, 4 centros; nível 2, 16 centros, servidos 4 de cada vez.

Outras estratégias de *search*: amostragem aleatória, hipercubo latino (LHS), sequências de Sobol e de Halton. O GLODS não é os 2^n -centros: é o enquadramento *search/poll* com proximidade; o *search* escolhe-se.

O GLODS completo



O *poll* assegura a convergência (teoria); o *search* procura bacias novas; a regra de comparação evita repetir trabalho.

Algoritmo (versão didática)

GLODS: $X, \alpha_0, r_0 \geq d_{\max}\alpha_0, \alpha_{\min}, N_{\max}$

```
1: lista  $L \leftarrow$  search inicial ( $2^n$ -centros de nível 0), cada ponto com  $\alpha_0, r_0$ , após a regra de comparação
2: while  $n_f < N_{\max}$  do
3:    $C \leftarrow \{x \in L \text{ ativo} : \alpha_x > \alpha_{\min}\}$   $\triangleright$  ativos ainda não identificados como mínimos
4:   if  $|C| \leq 1$  then search:  $2^n$  centros novos em  $L$ ; se algum entrou ativo, continue; senão recalcular  $C$ 
5:   end if
6:   if  $C = \emptyset$  then continue (próximo search) ou terminar, se níveis/orçamento esgotados
7:   end if
8:    $x \leftarrow \arg \min_C f$   $\triangleright$  centro de poll: o melhor ainda por explorar
9:   poll: para cada  $d \in D$ , submeter  $x + \alpha_x d$  à regra de comparação (passo  $\alpha_x$ , raio  $\alpha_x \|d\|$ )
10:  if entrou algum ativo then bem-sucedida:  $\alpha \leftarrow \gamma\alpha$  nos novos ativos
11:  else if entrou algum inativo then fusão: nada muda
12:  else sem sucesso:  $\alpha_x \leftarrow \beta\alpha_x$ 
13:  end if
14: end while
15: return ativos com  $\alpha \leq \alpha_{\min}$ , ordenados por  $f$   $\triangleright$  minimizantes locais
```

- A saída é uma **lista**, não um ponto: o primeiro é o melhor mínimo encontrado. O custo é n_f ; o orçamento $N_{\max}$ é o que o utilizador controla.
- $r =$ passo com que o ponto entrou ($\alpha \|d\|$; α com direções unitárias); não diminui quando α diminui ($r \geq \alpha$). No artigo, $r_0 \geq d_{\max}\alpha_0$.
- Determinístico com 2^n -centros; estocástico com amostragem aleatória (então: várias corridas, mediana).
- Versão didática: decréscimo simples, sem ordenação gulosa da lista nem amostragem em linha inicial; a estrutura é a do artigo.

O que se pode garantir?

Teorema (subsucessões refinantes) — Custódio e Madeira (2015)

Sob as hipóteses da estratégia de globalização usada (malhas inteiras com decréscimo simples, ou decréscimo suficiente com *forcing function*), X compacto e f limitada inferiormente: $\liminf \alpha_k = 0$, e existe uma subsucessão convergente de iterandos de *polls* sem sucesso com $\alpha_k \rightarrow 0$. (Globalização no sentido do 3.3.1: de qualquer x_0 , rumo à estacionariedade, não ao mínimo global.)

Teorema (estacionariedade)

Se f é L -Lipschitz (R) perto do limite x_* e as direções refinantes são densas no cone tangente, x_* é **estacionário no sentido de Clarke**: $f^\circ(x_*; d) \geq 0$ nas direções do cone tangente a X . Se f é **estritamente** diferenciável em x_* : no interior $\nabla f(x_*) = 0$; nos limites, KKT. Com $D_\oplus$ fixo (como nos exemplos) só se garante $f^\circ(x_*; d) \geq 0$ para $d \in D_\oplus$; com diferenciabilidade estrita isso basta. Para f só Lipschitz são precisas direções densas.

$f^\circ(x; d) = \limsup_{y \rightarrow x, t \downarrow 0} [f(y + td) - f(y)]/t$: derivada direcional generalizada de Clarke. Cone tangente a X em x : as direções pelas quais se entra em X a partir de x . Na linha de Torczon, Audet e Dennis, Kolda–Lewis–Torczon; apresentação completa.

O que isto diz — e o que não diz

- A teoria fornece **condições de estacionariedade para pontos limite de subsucessões refinantes**; aplicar o método não exige suavidade (só o teorema exige Lipschitz local).
- A **exploração global** vem do *search*: quantas bacias se encontram depende do orçamento, da amostragem e do raio de comparação. Sem hipóteses adicionais sobre f , não há garantia geral de identificar exatamente o mínimo global com um orçamento finito.
- Ruído: o GLODS não precisa de derivadas, que o ruído tornaria pouco fiáveis; mas esta *não* é uma teoria para funções ruidosas.

O que mostrou o artigo de 2015

61 problemas com limites, $2 \leq n \leq 10$, orçamento 20 000, paragem $\alpha < 10^{-8}$; perfis de dados contra **DIRECT**, **MCS** e um multiarranque puro. A conclusão é equilibrada:

- multiarranque puro: mau desempenho (o desperdício do início do deck); DIRECT atrás de MCS e GLODS;
- orçamentos *pequenos*: MCS foi o melhor no conjunto testado;
- GLODS **competitivo**, com o melhor desempenho para orçamentos *moderados*; com inicialização aleatória, a vantagem de incluir *search* tornou-se clara — 2^n -centros, Halton e Sobol com bom desempenho;
- e a característica adicional: procurar **os diferentes mínimos locais**, não só o melhor valor.

Problema (Tab. 6)	n	mín. locais co-nhecidos	encontrados
Becker–Lago	2	4	4
Branin–Hoo	2	3	2–3
Aluffi–Pentini	2	2	1
Shekel-45	4	5	4–5
Six-hump camel	2	6	2
Periodic	2	50	6–8

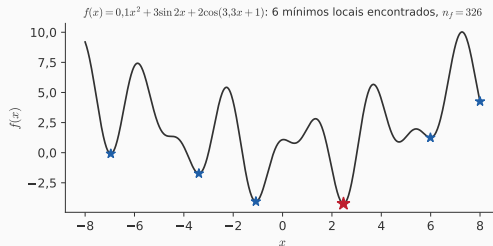
Variantes 2^n -centros, Sobol, Halton, com $\alpha_0 = r_0 = 1$ nos domínios originais. O GLODS procura explorar diferentes bacias; orçamento, *search* e raio de comparação determinam quantas consegue identificar. Em Aluffi–Pentini o artigo encontrou 1 dos 2; no exemplo didático, variar r_0 ($r_0 = 1$ dá 1, $r_0 = 0,25$ dá 2) mostra como o raio pode influenciar a separação das bacias.

GLODS na função do capítulo 5

$f(x) = 0,1x^2 + 3\sin 2x + 2\cos(3,3x + 1)$ em $[-8, 8]$;
 2^n -centros até ao nível 4, $\alpha_0 = r_0 = 1$, $\alpha_{\min} = 10^{-4}$,
 $n_f = 326$:

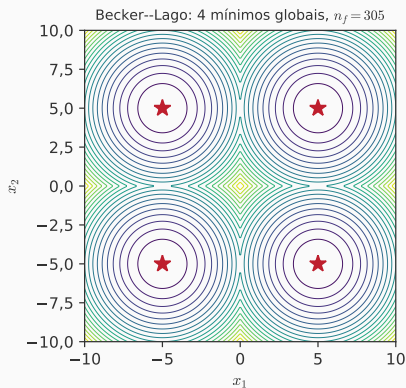
	x	$f(x)$
1	2,4680	-4,2381
2	-1,0819	-4,0531
3	-3,3918	-1,7273
4	-6,9615	-0,0849
5	5,9908	1,2485
6	8,0000	4,2533

- Os mínimos em 0,47 e 4,72 foram **absorvidos**: pontos dessas bacias caíram no raio de comparação de vizinhos com f menor. Não decide só a profundidade (4,72 é mais baixo do que 5,99): contam também a ordem e o raio.



- Cinco dos sete mínimos locais interiores, com o global $x^* = 2,468$, $f^* = -4,238$ **em primeiro**.
- $x = 8$: mínimo de fronteira (f desce até ao limite; estacionário no sentido Clarke-KKT).

Vários mínimos globais: Becker–Lago



$$f = (|x_1| - 5)^2 + (|x_2| - 5)^2 \text{ em } [-10, 10]^2.$$

- Quatro mínimos globais, todos com $f = 0$, em $(\pm 5, \pm 5)$.
- O GLODS encontra os quatro com $n_f = 305$ ($\alpha_0 = r_0 = 2$) — como no artigo (4 de 4).
- f é não diferenciável nos eixos: o algoritmo aplica-se sem derivadas; a análise de Clarke é o enquadramento adequado para interpretar a estacionariedade.
- Um método que devolve um só ponto daria um dos quatro — e não saberíamos que há outros três.

No computador: o código MATLAB do GLODS

O código original (versão 0.2) é distribuído na aula. Não é para ler de uma ponta à outra: o essencial é reconhecer nele as peças do método.

Ideia do método	Ficheiro
<i>search</i> (componente global)	<code>search_step.m</code>
proximidade: raio de comparação e lista de ativos/inativos	<code>match_point.m</code> , <code>merge.m</code>
algoritmo principal (<i>search/poll</i> , atualização de α)	<code>glods.m</code>
opções (estratégia de <i>search</i> , α_0 , r_0 , paragem, orçamento)	<code>parameters_glods.m</code>
ponto de partida: uma chamada simples num problema de teste	<code>driver_glods.m</code>

- Começar pelo `driver_glods.m`: correr, mudar a estratégia de *search* e o raio inicial, e ver quantos mínimos aparecem na lista.
- Só depois abrir `glods.m`, com o fluxograma «O GLODS completo» ao lado.

Otimização da orientação das fibras num provete de compósito laminado (Monte, Infante, Madeira, Moleiro, 2017)

Decisão a orientação das fibras em cada uma das 8 camadas: ângulos em $[-85^\circ, 90^\circ]$, em incrementos de 5° (36 valores por camada).

Objetivo minimizar o deslocamento máximo do provete sob carga.

Informação sobre f só valores: cada avaliação é uma análise de elementos finitos (ANSYS), chamada pelo GLODS — caixa negra, cara, sem derivadas.

Resultado uma lista de 14 soluções com deslocamentos máximos entre 3,17 e 3,60 mm — todas resolvem o problema.

A decisão final não foi “a de menor f ”: entre soluções equivalentes em desempenho, **escolheu-se a mais fácil de fabricar (montar)**. Um método que devolvesse apenas uma solução não disponibilizaria naturalmente este conjunto de alternativas numa única execução. Referência: *Mechanics of Advanced Materials and Structures* 24(5):410–416, 2017, DOI 10.1080/15376494.2016.1191099.

O mínimo global é sempre a solução escolhida?

Duas das 14 soluções do compósito:

Solução 12 — o melhor f

$[-35/45/40/80/ - 40/50/ - 55/40]$

deslocamento máximo **3,17 mm**

oito orientações diferentes

Solução 1

$[45/ - 50/ - 50/45/45/ - 50/ - 50/45]$

deslocamento máximo **3,40 mm** (+7%)

só duas orientações, num padrão simétrico

O mínimo global é sempre a solução escolhida?

Duas das 14 soluções do compósito:

Solução 12 — o melhor f

$[-35/45/40/80/ - 40/50/ - 55/40]$

deslocamento máximo **3,17 mm**

oito orientações diferentes

Solução 1

$[45/ - 50/ - 50/45/45/ - 50/ - 50/45]$

deslocamento máximo **3,40 mm** (+7%)

só duas orientações, num padrão simétrico

Pergunta: qual preferiria montar?

O mínimo global é sempre a solução escolhida?

Duas das 14 soluções do compósito:

Solução 12 — o melhor f

$[-35/45/40/80/ - 40/50/ - 55/40]$

deslocamento máximo **3,17 mm**

oito orientações diferentes

Solução 1

$[45/ - 50/ - 50/45/45/ - 50/ - 50/45]$

deslocamento máximo **3,40 mm (+7%)**

só duas orientações, num padrão simétrico

Pergunta: qual preferiria montar?

O ótimo matemático é o **melhor segundo o modelo** que formulámos.

A decisão de engenharia pode usar **informação que não entrou no modelo**.

Se escolhermos outra solução por custo, fabrico ou materiais, isso revela que havia critérios relevantes fora de f . Se o algoritmo só tivesse devolvido a solução 12, nem saberíamos que a 1 existia.

Então porque encontrar vários mínimos?

Porque bacias diferentes podem corresponder a **alternativas de projeto qualitativamente diferentes**.

- **Desempenho próximo** → alternativas quase equivalentes.
- **Fabrico, custo, materiais** → critérios que não estão em f .
- **Robustez da solução** (não do método) → uma bacia larga (pequenas alterações de x pouco degradam f) pode ser preferível a um mínimo estreito ligeiramente melhor.
- **Decisão posterior** → o engenheiro escolhe entre alternativas, com a lista na mão.

Um algoritmo que devolve só o melhor ponto responde a “qual a melhor solução segundo f ?”. Uma lista responde também a “**que alternativas tenho?**”.

Então porque encontrar vários mínimos?

Porque bacias diferentes podem corresponder a **alternativas de projeto qualitativamente diferentes**.

- **Desempenho próximo** → alternativas quase equivalentes.
- **Fabrico, custo, materiais** → critérios que não estão em f .
- **Robustez da solução** (não do método) → uma bacia larga (pequenas alterações de x pouco degradam f) pode ser preferível a um mínimo estreito ligeiramente melhor.
- **Decisão posterior** → o engenheiro escolhe entre alternativas, com a lista na mão.

Um algoritmo que devolve só o melhor ponto responde a “qual a melhor solução segundo f ?”. Uma lista responde também a “**que alternativas tenho?**”.

E se esses critérios forem suficientemente importantes?

Talvez não devam ficar fora do modelo: tornam-se **objetivos**.

desempenho $\longrightarrow$ (desempenho, custo, fabricabilidade, ...)

⇒ **cap. 6: vários objetivos**.

Então porque encontrar vários mínimos?

Porque bacias diferentes podem corresponder a **alternativas de projeto qualitativamente diferentes**.

- **Desempenho próximo** → alternativas quase equivalentes.
- **Fabrico, custo, materiais** → critérios que não estão em f .
- **Robustez da solução** (não do método) → uma bacia larga (pequenas alterações de x pouco degradam f) pode ser preferível a um mínimo estreito ligeiramente melhor.
- **Decisão posterior** → o engenheiro escolhe entre alternativas, com a lista na mão.

Um algoritmo que devolve só o melhor ponto responde a “qual a melhor solução segundo f ?”. Uma lista responde também a “**que alternativas tenho?**”.

E se esses critérios forem suficientemente importantes?

Talvez não devam ficar fora do modelo: tornam-se **objetivos**.

desempenho $\longrightarrow$ (desempenho, custo, fabricabilidade, ...)

⇒ **cap. 6: vários objetivos**.

E com vários objetivos?

O **MultiGLODS** estende esta ideia global–local à otimização multiobjetivo (Custódio e Madeira, *J. Global Optim.* 72, 2018). Não o estudamos nesta UC; voltamos a vários objetivos no cap. 6.

5.1 GLODS — quadro de consulta

Que informação temos sobre f ? Quanto custa obtê-la? O que conseguimos garantir com ela?

Informação	apenas valores de f (ordem 0); limites $\mathbf{x}^{(L)} \leq \mathbf{x} \leq \mathbf{x}^{(U)}$ por barreira extrema
Exploração global	várias pesquisas diretas locais (<i>poll</i>) + <i>search</i> em zonas por explorar, sem duplicar pesquisas próximas: raio de comparação e lista de ativos/inativos
Custo	avaliações de f : $n_f \leq D $ por iteração ($2n$ com $[I - I]$) mais as do <i>search</i> ; o orçamento $N_{\max}$ é a decisão do utilizador
Garantia	estacionariedade (Clarke) dos pontos limite de subsucessões refinantes, se f for Lipschitz perto do limite e as direções refinantes forem densas (com $D_{\oplus}$ fixo: só nas direções de $D_{\oplus}$); sem garantia de encontrar todas as bacias com n_f finito
Saída	uma <i>lista</i> de candidatos a minimizantes locais, ordenada por f ; permite decidir por critérios que não estavam em f
Aleatoriedade	pode ser determinístico (2^n -centros; sequências determinísticas) ou estocástico (amostragem aleatória, LHS); neste caso depende da semente e requer várias corridas
Limitação	dimensão elevada ($2n$ sondagens por iteração, 2^n -centros) e funções muito caras exigem gerir o orçamento; bacias pequenas podem ser absorvidas pelo raio

O QUE DEVE FICAR — 5.1

1. **Exploração global:** *poll* local + *search* em zonas novas; o raio de comparação evita repetir pesquisas.
2. **Informação e custo:** só valores de f ; custo em avaliações de f , com orçamento fixado.
3. **Garantia:** estacionariedade (Clarke) dos limites refinantes, se f for Lipschitz perto do limite e as direções refinantes densas (com $D_{\oplus}$ fixo: só nas direções de $D_{\oplus}$); nenhuma garantia de encontrar todas as bacias.
4. **Saída:** uma **lista** de candidatos a minimizantes locais — para escolher também por critérios fora de f .
5. **Aleatoriedade:** determinístico com *search* determinístico; senão, várias corridas.

A seguir (5.2, 5.3): aleatoriedade — o SA aceitando subidas, o AG com uma população.

Referências Lista completa: *Bibliografia da UC*, no fim do PDF da UC.

- Custódio, A. L., Madeira, J. F. A., *GLODS: Global and Local Optimization using Direct Search*, Journal of Global Optimization 62(1):1–28, 2015 — o artigo do GLODS.
- Conn, A. R., Scheinberg, K., Vicente, L. N., *Introduction to Derivative-Free Optimization*, SIAM (MPS-SIAM Series on Optimization 8), 2009 — cap. 7.
- Madeira, J. F. A., *Multiobjective and global optimization with directional direct search*, Tese de Doutoramento em Matemática, IST, Universidade de Lisboa, 2019.
- Horst, R., Tuy, H., *Global Optimization: Deterministic Approaches*, 3.^a ed., Springer, 1996.

Recursos

- Extensões: Madeira (2026) (GLODS-SI, invariância de escala); *GLODS-SI-Mix* — variáveis contínuas, discretas e categóricas em simultâneo (submetido).
- A **apresentação completa** (98 páginas): convergência com as derivadas de Clarke–Jahn, perfis de dados contra DIRECT e MCS. Estes apontamentos são a versão didática.
- GLODS v0.2 (MATLAB), distribuído na aula com o relatório técnico (ficheiros em «No computador»); página: docentes.fct.unl.pt/algb/pages/glods.
- Dissertações no IST: Luís (2022, GLODS e SA), Figueiredo e Mareco (2016) — *Bibliografia da UC*.